

Ruminations on explicit multi-param constructors of views

Abstract

This paper is about [LWG 3714](#). Non-single-argument constructors for range adaptors should not be explicit.

We have C++20 views, none of which have explicit multi-param constructors, and some newer C++23 views which do. This is an obscure and rarely-noticeable difference. This paper looks at some aspects of it.

Spoiler

SG9 decided to go for making the C++20 views' constructors explicit as well, even though that's a breaking change.

POLL: We support applying "LWG3714: Non-single-argument constructors for range adaptors should not be explicit" AKA option 2 in Ville's paper:

SF	F	N	A	SA
1	4	3	2	0

Despite keeping all the original rumination, this paper contains wording for doing exactly that. As an addition, while looking at the wording, I became quite puzzled why any of the `_single_`-argument constructors should be implicit, and that would be yet another inconsistency, so the wording in this paper fixes those too for the C++20 views.

Why do we care?

That's an interesting question. By and large, we don't, and we don't want to. Most if not all code should `_always_` use `views::foo`, instead of constructing a `foo_view`. I personally don't find the examples in the issue convincing at all, and from what I gather, neither does anyone else, much. But we have a difference, and whether it's explicable, necessary, or useful, is... somewhat questionable.

Let's dive right in, then. In the issue, we have an example thus:

```
std::ranges::chunk_view r1 = {v, 1}; // ill-formed
```

`chunk_view`'s multi-param constructor is explicit, so you can't do that. You can do it for a `filter_view`, for example, as mentioned in the issue, because there the constructor isn't explicit.

Okay, fine, one could write, instead, this:

```
std::ranges::chunk_view r1{v, 1}; // now it's fine
```

Cool. `_We_` all know that that's vastly different, the explicit constructor forces us to name the type when we initialize an object using an explicit constructor.. ..but to a naive reader, that's still just a one-character difference, "drop the '=' and it's fine".

For this particular example, I find it highly dubious what extra protection we're really providing with the explicit constructor.

Okay, fine, what about a function parameter? We could have an example like thus:

```
template <class V> void f(std::ranges::chunk_view<V> p);  
f({v, 1});
```

This is ill-formed regardless of whether the constructor is explicit, because template deduction from a plain braced-list doesn't happen. So in this case, an implicit constructor doesn't provide convenience, and an explicit constructor doesn't provide particular protection.

Okay, let's try hard. Let's try really really really hard to come up with a remotely explicable example, emphasis on "remotely":

```
#include <vector>  
#include <ranges>  
  
using my_filter_view = decltype(std::views::filter(std::declval<std::vector<int>&>(), std::declval<bool (*)(int)>()));  
  
void f(my_filter_view) {}  
  
int main() {  
    std::vector<int> v;  
    f({v, +[](int x) -> bool {return x % 2;}});  
}
```

Congratulations to me, here we have a non-template function taking a filter on a vector of ints, and we can pass in multiple different predicates, because the function takes a filter that uses a `bool (*)(int)`. Lambdas converted to function pointers work fine. We can even bake `f()` into an ABI, and now we can use unnamed/untyped brace-init. We couldn't do that if we used a `chunk_view` instead of `filter_view`, because there the constructor is explicit.

So _that's your convincing example?

No. :) Not at all. :) I wouldn't recommend to anyone to write something like that. I would recommend keeping view types out of ABIs, or at worst, using a type-erased view in an ABI. But something like that.. as I said, we're trying really hard, to come up with something `_remotely_` plausible. It required some serious effort, and the result isn't all that plausible, if you ask me.

However.

I find it similarly hard to come up with a plausible example where the explicit constructor is truly `_useful_`, so that it protects innocent users from making mistakes. As things are, it seems to require some serious effort to write code where that presumed protection ends up protecting an innocent user.

An attempt at a summary of thoughts

It sure seems to me that to be explicit or not

- certainly can be seen as supporting code that we wouldn't recommend anyone to write in the first place
- but it also doesn't seem to offer meaningful protection for code that we expect anyone to write

so we seem to... be protecting against mistakes that shouldn't ever occur to begin with.

So we certainly have a choice to make here:

1. Make the C++20 view multi-param constructors explicit too, if we believe the protection is worthwhile
2. Drop the explicit from the C++23 views, if we don't believe the protection is worthwhile.

To me, the whole issue seems super-obscure, and pedantic to the hilt. There seems to be various differences of opinion whether we should care at all, but I find it equally plausible to consider the different treatise of explicit in the constructors of different views just... weird. Yeah, sure, it's just an inconsistency. The practical impact should be extremely small. But in the same vein and in the same breath, that calls into question what the purpose and importance of using explicit where it wasn't used before really is.

I personally can't get excited over this either way. I could live with both adding explicit to the C++20 views, or with dropping it from the C++23 views. I don't think it's really explicable to have this sort of a difference, but I find it hard to believe it's a significant matter that should necessarily have high importance. In certain ways, I'd really prefer that we go either way soon, and be done with it.

Wording

Drafting Note: the intent here is to make every constructor that is not a special member function of every range view in the standard explicit.

Including single-parameter constructors. The original discussion was about multi-parameter constructors, but we want API design consistency here, and don't want to introduce a different inconsistency while fixing another.

--End Drafting Note.

In `[range.iota.view]` synopsis, add explicit:

```
constexpr explicit iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
constexpr explicit iota_view(iterator first, see below last);
```

Before `[range.iota.view]/8`, add explicit:

```
constexpr explicit iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
```

Before `[range.iota.view]/10`, add explicit:

```
constexpr explicit iota_view(iterator first, see below last);
```

In `[range.ref.view]/1`, add explicit:

```
constexpr explicit ref_view(T&& t);
```

Before `[range.ref.view]/2`, add explicit:

```
constexpr explicit ref_view(T&& t);
```

In `[range.owning.view]/1`, add explicit:

```
constexpr explicit owning_view(R&& t);
```

Before `[range.owning.view]/2`, add explicit:

```
constexpr explicit owning_view(R&& t);
```

In `[range.filter.view]` synopsis, add explicit:

```
constexpr explicit filter_view(V base, Pred pred);
```

Before `[range.filter.view]/1`, add explicit:

```
constexpr explicit filter_view(V base, Pred pred);
```

In `[range.transform.view]` synopsis, add explicit:

```
constexpr explicit transform_view(V base, F fun);
```

Before `[range.transform.view]/1`, add explicit:

```
constexpr explicit transform_view(V base, F fun);
```

In `[range.take.view]` synopsis, add explicit:

```
constexpr explicit take_view(V base, range_difference_t count);
```

Before [range.take.view]/1, add explicit:

```
constexpr explicit take_view(V base, range_difference_t count);
```

In [range.take.while.view] synopsis, add explicit:

```
constexpr explicit take_while_view(V base, Pred pred);
```

Before [range.take.while.view]/1, add explicit:

```
constexpr explicit take_while_view(V base, Pred pred);
```

In [range.drop.view] synopsis, add explicit:

```
constexpr explicit drop_view(V base, range_difference_t count);
```

Before [range.drop.view]/1, add explicit:

```
constexpr explicit drop_view(V base, range_difference_t count);
```

In [range.drop.while.view] synopsis, add explicit:

```
constexpr explicit drop_while_view(V base, Pred pred);
```

Before [range.drop.while.view]/1, add explicit:

```
constexpr explicit drop_while_view(V base, Pred pred);
```

In [range.join.with.view] synopsis, add explicit:

```
constexpr explicit join_with_view(V base, Pattern pattern);  
...  
constexpr explicit join_with_view(R&& r, range_value_t<InnerRng> e);
```

Before [range.join.with.view]/1, add explicit:

```
constexpr explicit join_with_view(V base, Pattern pattern);
```

Before [range.join.with.view]/2, add explicit:

```
constexpr explicit join_with_view(R&& r, range_value_t<InnerRng> e);
```

In [range.lazy.split.view] synopsis, add explicit:

```
constexpr explicit lazy_split_view(V base, Pattern pattern);  
...  
constexpr explicit lazy_split_view(R&& r, range_value_t<R> e);
```

Before [range.lazy.split.view]/1, add explicit:

```
constexpr explicit lazy_split_view(V base, Pattern pattern);
```

Before [range.lazy.split.view]/2, add explicit:

```
constexpr explicit lazy_split_view(R&& r, range_value_t<R> e);
```

In [range.split.view] synopsis, add explicit:

```
constexpr explicit split_view(V base, Pattern pattern);  
...  
constexpr explicit split_view(R&& r, range_value_t<R> e);
```

Before [range.split.view]/1, add explicit:

```
constexpr explicit split_view(V base, Pattern pattern);
```

Before [range.split.view]/2, add explicit:

```
constexpr explicit split_view(R&& r, range_value_t<R> e);
```